

Project Report

DRIVER MONITORING SYSTEM DATA ANALYSIS TOOL

Presented By

- 1) Prajwal Phalke**
- 2) Vanshita Sonule**
- 3) Afreen Ansari**

1. Project Title

Driver Monitoring System (DMS) Data Analysis Tool

DMS is a Python tool that reads driver-behaviour sensor data and automatically generates a driver safety report.

2. Project Overview

What it does: DMS reads eye closure, blink rate, head pitch and yawning readings for 5 drivers from an Excel file, calculates an Attention Score for every driver in every shift, ranks the drivers from safest to riskiest, and generates a Word (and PDF) report of the results.

Purpose: To replace manual checking of driver fatigue data with an automatic, consistent, and easy-to-share safety report — useful for a fleet manager to quickly see which driver or shift needs attention.

The project has 5 Python files working together:

- **main.py** — runs the whole program, step by step.
- **loader.py** — reads and validates the Excel dataset.
- **evaluator.py** — calculates the weighted attentiveness scores and ranking.
- **report.py** — builds the Word report with tables and charts.
- **word_to_pdf.py** — converts the finished Word report into a PDF.

3. Project Flow Diagram

The diagram below shows the step-by-step execution flow, from running main.py to the final Word + PDF report being saved on disk.



4. Project Execution — File-by-File Breakdown

This section documents every source file: its purpose, its functions, and what each one does. All descriptions are based on a direct reading and an actual dry run of the provided code.

4.1 main.py — Entry Point / Orchestrator

Purpose: The file the user runs (python main.py). Prepares folders, triggers backup, runs the scoring pipeline, prints results, and builds the Word + PDF report.

Function	What it does
create_folders()	Creates the data/, reports/ and backup/ folders if they don't exist.
backup_dataset(file_path)	Copies the Excel dataset into backup/ before it's used, skipping quietly if the file is missing or locked.
show_driver_results(ranking)	Prints a clean console table of all 5 drivers' overall score/status, then highlights the single best driver.
main()	Runs everything in order: folders → backup → load data → score drivers → rank → print results → build Word report → convert it to PDF → print file locations.

Step-by-Step Working

- Step 1: Create the data/, reports/ and backup/ folders if they don't already exist.
- Step 2: Back up the Excel dataset into the backup/ folder.
- Step 3: Load the dataset — stop the program here if it fails to load.
- Step 4: Calculate the weighted Attention Score for every driver/shift.
- Step 5: Rank all 5 drivers by their overall score.
- Step 6: Print the ranking table and the best driver to the console.
- Step 7: Build the Word report (tables + 19 charts) and save it.
- Step 8: Convert the saved Word report into a matching PDF.
- Step 9: Print the Word and PDF file locations — 'PROJECT COMPLETED'.

If/Else Conditions Used in main.py

Condition (in code)	Purpose
if not os.path.exists(file_path)	Skip the backup step if the dataset file isn't found.
try / except PermissionError	Avoid crashing if the backup file is locked/open.
if data is None	Stop the program if the dataset failed to load.
if pdf_report is not None	Only print a PDF location if the PDF was actually created.
if __name__ == "__main__"	Run main() only when the file is executed directly.

4.2 modules/loader.py — DataLoader class

Purpose: Safely reads and validates the Excel dataset before anything else runs.

Function	What it does
<code>show_excel_sheets()</code>	Opens the workbook and prints the names of every sheet it contains.
<code>load_data()</code>	Checks the file exists, reads the 'Driver Monitoring Data' sheet, checks that all 9 required columns are present, converts the 6 numeric columns properly (turning bad/text values into blanks), fills any remaining blanks with 0, then returns the cleaned data.

If/Else Conditions Used in `load_data()`

Condition (in code)	Purpose
<code>if not os.path.exists(self.file_path)</code>	Stop and report an error if the Excel file itself is missing.
<code>if missing (required columns check)</code>	Stop if any of the 9 expected columns aren't in the sheet — prevents the rest of the program from crashing later.
<code>pd.to_numeric(..., errors="coerce")</code>	Any non-numeric/corrupt reading in a sensor column becomes blank instead of crashing the load.
<code>data[numeric] = data[numeric].fillna(0)</code>	Any remaining blank sensor readings are treated as 0 rather than being dropped.
<code>try / except Exception</code>	Catches any other read error and reports it instead of crashing.

4.3 modules/evaluator.py — DriverEvaluator class

Purpose: The scoring engine. Converts the 4 raw sensor readings into a single weighted 0–100 Attention Score per driver/shift, then ranks all drivers.

Function	What it does
<code>eye_score / blink_score / head_score / yawn_score</code>	Each converts one raw sensor reading into its own 0–100 sub-score using fixed value ranges (see formula below).
<code>status(score)</code>	Converts a score into a label: Attentive (≥ 85), Moderately Attentive (≥ 70), or Inattentive (below 70).
<code>recommendation(status)</code>	Returns a one-line suggestion matching the status (e.g. 'Take a short break').
<code>calculate_row_scores()</code>	Applies all 4 sub-score functions to every row, then combines them into the weighted Attention_Score for that row.
<code>calculate_scores()</code>	Groups the scored rows by driver + shift and averages them into one row per driver/shift, with a Status and Recommendation.

Function	What it does
<code>create_ranking(driver_report)</code>	Averages each driver's shifts into one Overall_Score, sorts drivers from best to worst, and assigns Rank 1–5.

Scoring Formula Used

Every reading is first converted into its own 0–100 sub-score, using fixed thresholds:

```
def eye_score(self, value):
    if value <= 5:
        return 100
    if value <= 10:
        return 85
    if value <= 15:
        return 65
    if value <= 20:
        return 40
    return 15

def blink_score(self, value):
    if 12 <= value <= 20:
        return 100
    if 10 <= value < 12:
        return 75
    if 8 <= value < 10:
        return 50
    if value < 8:
        return 20
    if value <= 25:
        return 80
    return 50
```

Actual code — Eye Closure and Blink Rate sub-scores (modules/evaluator.py)

- **Why `eye_score` is written this way:** eye closure % is the most direct sign of drowsiness — the longer the eyes stay shut, the closer the driver is to falling asleep. So the code scores it in falling steps (5/10/15/20%): a small closure barely matters, but every step past that drops the score faster, since risk grows quickly once eyes start staying shut.
- **Why `blink_score` is written this way:** a healthy driver blinks at a steady 12–20 times/minute. Blinking much slower is a classic drowsiness sign, so it's scored low. But blinking too fast can mean strain or distraction, so very high rates aren't scored perfectly either — this is why the code checks a range (12–20) instead of just one direction.

```
def head_score(self, value):
    value = abs(value)
    if value <= 5:
        return 100
    if value <= 10:
        return 80
    if value <= 15:
        return 55
    return 20

def yawn_score(self, value):
    if value <= 0:
        return 100
    if value == 1:
        return 65
    return 20
```

Actual code — Head Pitch and Yawning sub-scores (modules/evaluator.py)

- **Why head_score is written this way:** a driver's head tilts down or to the side when they're nodding off, so `abs(value)` is used — the direction of the tilt doesn't matter, only how far off-centre the head is. Bigger tilt = more likely they're dozing, so the score drops faster the more it exceeds normal posture.
- **Why yawn_score is written this way:** yawning is a strong, unmistakable fatigue signal, so the score is written to fall sharply — even a single yawn (score 65) is treated as a real warning, not a minor deduction, because by the time a driver yawns, fatigue is already noticeable.

The 4 sub-scores are then combined using fixed weights to get the final Attention Score, which decides the status:

```
# Weighted Attention Score
Attention_Score = (
    Eye_Score * 0.35
    + Blink_Score * 0.25
    + Head_Score * 0.25
    + Yawning_Score * 0.15)

def status(self, score):
    if score >= 85:
        return "Attentive"
    if score >= 70:
        return "Moderately Attentive"
    return "Inattentive"
```

Actual code — Final weighted score and status (modules/evaluator.py)

Why these particular weights: not every reading is an equally reliable sign of fatigue, so the code doesn't treat them equally.

- **Eye Closure — 35% weight** — the biggest factor; it's the single most reliable physical sign of drowsiness, so it drives the score the most.
- **Blink Rate — 25% weight** — a strong supporting signal to eye closure, so it's weighted second-highest alongside Head Pitch.
- **Head Pitch — 25% weight** — equally important, since a drooping head is an easy, visible confirmation of fatigue.
- **Yawning — 15% weight** — kept lowest only because yawning happens less often in the data than the other 3 readings, even though each yawn is treated harshly by `yawn_score` itself.

Unlike a penalty system, every reading always contributes something — there's no '0 deduction' case, so the final score is a smooth blended average, not a start-at-100-and-subtract number. This is why the formula reads as an addition of weighted parts rather than 100 minus a list of penalties.

4.4 modules/report.py — WordReport class

Purpose: The presentation layer. Turns the scored data into a fully formatted Word document with a running header/footer, 9 tables, and 19 charts.

Function	What it does
<code>__init__</code> / <code>prepare_status_data()</code>	Sets up the document (header, footer, font), and splits each shift's score into an <code>Attentiveness_Score</code> (Eye+Head) and <code>Alertness_Score</code> (Blink+Yawn) for extra detail.
<code>cover_page()</code> / <code>executive_summary()</code> / <code>dashboard()</code>	Build the title page, a one-line scoring summary, and a dashboard table of fleet-wide totals/averages.
<code>all_driver_shift_status()</code>	Builds one big table listing every driver's every shift with <code>Attentiveness</code> , <code>Alertness</code> and <code>Overall</code> scores/status side by side.
<code>driver_summary()</code>	Builds the final ranking table (Rank, Driver, Overall, Status, and all 4 component scores).
<code>create_overall_charts()</code>	Generates the 4 fleet-wide charts: ranking bar, driver comparison bar, all-drivers-all-shifts line chart, and component comparison chart.
<code>overall_driver_comparison()</code>	Adds a best/worst driver & shift summary table, then embeds the 4 fleet-wide charts.
<code>create_driver_charts()</code> / <code>driver_analysis()</code>	For every driver: builds a per-shift table plus 3 charts (score trend, component scores, raw sensor trend), one dedicated subsection each.
<code>conclusion()</code>	Auto-writes a closing paragraph naming the best/worst driver and shift, using the real numbers from this run.
<code>generate_report()</code>	Runs all the above in order, backs up any existing report first, saves the .docx, deletes temporary chart files, and returns the file path.

4.5 word_to_pdf.py — PDF Export

Purpose: Takes the finished Word report and exports a matching PDF, using Microsoft Word itself via Windows automation (pywin32).

Function	What it does
<code>convert_word_to_pdf(word_file)</code>	Opens the Word file in the background using MS Word, exports it as a PDF next to the original, then closes Word again.

If/Else Conditions Used

Condition (in code)	Purpose
<code>if not os.path.isfile(word_path)</code>	Stop if the Word file to convert can't be found.
<code>except ImportError</code>	If pywin32 isn't installed (e.g. on Mac/Linux), print instructions instead of crashing.
<code>except Exception</code>	Catches any other conversion error (e.g. Word not installed) and reports it safely.
<code>finally: close Word / quit application</code>	Always closes the Word document and application, even if the conversion failed.

Note: this only works on Windows with Microsoft Word installed. On Mac/Linux (or without pywin32), the program still completes normally — it just skips the PDF and keeps the Word file.

4.6 Supporting Files

File / Folder	Purpose
<code>data/DMS_Final_Workbook_Final.xlsx</code>	The source dataset — 300 rows across 5 drivers, read from its 'Driver Monitoring Data' sheet.
<code>backup/</code>	Auto-filled with a fresh dataset copy every run, plus a timestamped copy of the previous Word report each time a new one is generated.
<code>reports/</code>	Holds the current <code>Analysis_Report.docx</code> (and its <code>.pdf</code> , when PDF export succeeds).
<code>modules/analyzer.py</code>	Legacy file — still contains a <code>DataAnalyzer</code> class, but it is no longer imported or called anywhere in the current project.
<code>modules/__pycache__/, __pycache__/</code>	Auto-generated compiled Python bytecode; not source code, safe to ignore/delete.

5. Program Output — Dry Run Results

The project was run successfully on the real dataset (300 records, 5 drivers). It produces 3 outputs: console summary text, a backup Excel file, and a Word report (plus a PDF, on Windows).

5.1 Console Output

On loading, the program confirms the sheet names and record count, then prints the driver ranking straight away:

Rank	Driver	Overall Score	Status
1	Rahul	100.00	Attentive
2	Priya	97.02	Attentive
3	Aman	93.95	Attentive
4	Neha	86.16	Attentive
5	Arjun	63.43	Inattentive

Result: Rahul is the best driver overall (100.00/100, Attentive). Arjun is the only Inattentive driver, mainly dragged down by a poor Night shift.

5.2 Excel Backup

A copy of the dataset is auto-saved to backup/DMS_Dataset_Backup.xlsx on every run, so the original file is never lost.

5.3 Word Report (reports/Analysis_Report.docx)

The main deliverable — a formatted report with tables and charts. Sample pages from the actual generated file:

Driver Monitoring System Analysis Report

1. Executive Summary

The score uses Eye Closure (35%), Blink Rate (25%), Head Pitch (25%) and Yawning (15%). Results update when the Excel data changes.

2. Dashboard Summary

Total Records 300	Total Drivers 5
Average Eye Closure 8.08%	Average Blink Rate 12.93
Average Calculated Score 88.11	Inattentive Records 35

Sample output — Executive Summary and Dashboard Summary

3. All Drivers and All Shifts Status

Driver	Shift	Attentiveness	Attention Status	Alertness	Alertness Status	Overall	Overall Status
Aman	Morning	93.1	Attentive	97.7	Alert	94.9	Attentive
	Afternoon	89.2	Attentive	97.7	Alert	92.6	Attentive
	Night	93.1	Attentive	96.1	Alert	94.3	Attentive
Arjun	Morning	77.2	Moderately Attentive	97.7	Alert	85.4	Attentive
	Afternoon	59.7	Inattentive	73.2	Moderately Alert	65.1	Inattentive
	Night	43.3	Inattentive	34.6	Drowsy Risk	39.9	Inattentive
Neha	Morning	88.2	Attentive	100.0	Alert	92.9	Attentive
	Afternoon	82.3	Moderately Attentive	97.9	Alert	88.5	Attentive
	Night	75.6	Moderately Attentive	79.2	Moderately Alert	77.0	Moderately Attentive
Priya	Morning	98.7	Attentive	100.0	Alert	99.2	Attentive
	Afternoon	95.2	Attentive	100.0	Alert	97.1	Attentive
	Night	92.1	Attentive	98.7	Alert	94.8	Attentive
Rahul	Morning	100.0	Attentive	100.0	Alert	100.0	Attentive
	Afternoon	100.0	Attentive	100.0	Alert	100.0	Attentive
	Night	100.0	Attentive	100.0	Alert	100.0	Attentive

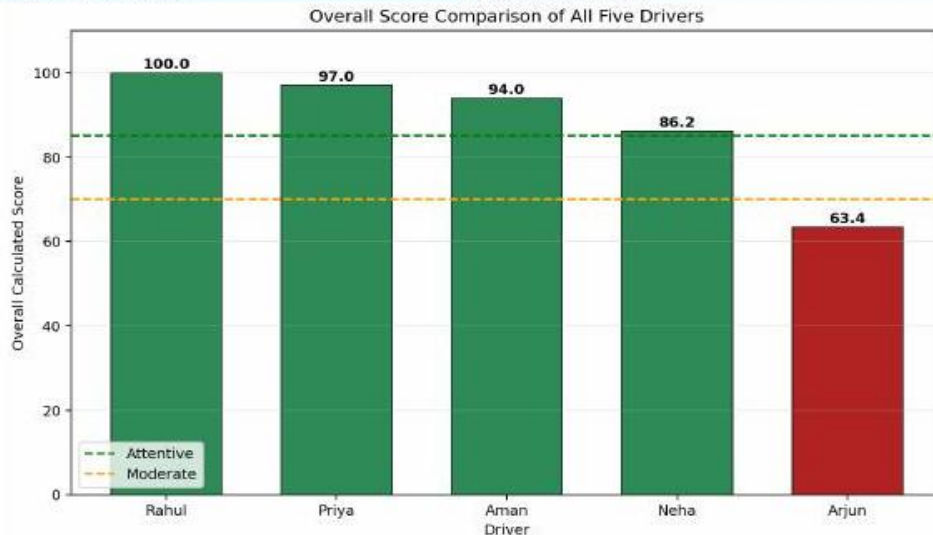
4. Driver Performance Summary

Rank	Driver	Overall	Status	Eye	Blink	Head	Yawn
1	Rahul	100.00	Attentive	100.0	100.0	100.0	100.0
2	Priya	97.02	Attentive	92.0	100.0	100.0	98.8

Sample output — All Drivers and All Shifts Status, and Driver Performance Summary

5. Overall Driver and Shift Comparison

Best Overall Driver	Rahul (100.00/100)
Best Driver Status	Attentive
Best Overall Shift	Morning (94.48/100)
Needs Most Attention	Arjun (63.43/100)



Sample output — Overall Driver and Shift Comparison charts

6. Chart & Table Reference — What Each Output Specifies

A short explanation of what each table and chart in the report tells you.

6.1 Data Tables

Table	What it tells you
Dashboard Summary	Fleet-wide totals and averages — records, drivers, eye closure, blink rate, score, inattentive count.
All Drivers and All Shifts Status	Every driver's every shift, with Attentiveness, Alertness and Overall scores side by side.
Driver Performance Summary (Ranking)	Ranks all 5 drivers from best to worst by Overall Score.
Overall Driver and Shift Comparison	Names the single best/worst driver and best/worst shift in one glance.
Per-Driver Shift Table	One driver's full breakdown across Morning/Afternoon/Night, with every component score.

6.2 Charts in the Word Report

Chart	What it tells you
Overall Score Bar Comparison	Quick leaderboard of all 5 drivers' overall scores, colour-coded by status.

Chart	What it tells you
Overall Driver Attentiveness Ranking (horizontal bar)	Same ranking, sorted top to bottom for easy reading.
All Drivers and All Shifts (line chart)	Shows every driver's score trend across the day in one chart — easiest way to spot who declines by Night.
Component Comparison (bar chart)	Compares each driver's 4 sub-scores (Eye/Blink/Head/Yawn) side by side.
Calculated Attention Score by Shift (per driver)	One driver's score trend across their 3 shifts.
Calculated Component Scores (per driver)	One driver's 4 sub-scores plotted across their shifts.
Sensor Parameter Trend (per driver)	The driver's raw, un-scored sensor readings across shifts, for reference.

7. Libraries Used & Their Purpose

Every library actually imported in the code, where it's used, and why.

Library	Used in	Purpose in this project
pandas	loader.py, evaluator.py, report.py	Reads the Excel sheet, cleans the numeric columns, and powers every groupby/aggregation used to build the scores and ranking.
numpy	evaluator.py	Used to detect and round the numeric columns in the ranking table.
openpyxl	loader.py	The engine pandas uses to read the .xlsx file and list its sheet names.
matplotlib (pyplot)	report.py	Draws all 19 charts — the 4 fleet-wide charts and the 3 per-driver charts × 5 drivers.
python-docx (docx)	report.py	Builds the Word document: headings, header/footer, styled tables, and embeds every chart image.
pywin32 (win32com.client)	word_to_pdf.py	Automates Microsoft Word (Windows only) to export the finished report as a PDF. Optional — the program still works without it.
os	main.py, loader.py, report.py, word_to_pdf.py	Standard library — creates folders and checks whether files exist before acting on them.
shutil	main.py, report.py	Standard library — copies the dataset and old reports into backup/.

Library	Used in	Purpose in this project
datetime	report.py	Standard library — builds the timestamped backup filenames.

Three groups: pandas/numpy/openpyxl handle data loading and scoring, matplotlib handles the 19 charts, and python-docx + pywin32 (with os/shutil/datetime) turn everything into a shareable Word and PDF report.

8. Conclusion

The Driver Monitoring System successfully automates the full journey from raw sensor readings to a decision-ready safety report. It reads eye closure, blink rate, head pitch and yawning data for 5 drivers, converts each reading into a weighted 0–100 Attention Score (35% Eye, 25% Blink, 25% Head, 15% Yawning), and ranks every driver from safest to riskiest — with no manual spreadsheet work needed.

The code was dry-run in full while preparing this documentation and completed without errors, correctly scoring all 300 records for 5 drivers and producing a 19-chart Word report. On this run, Rahul was the safest driver (100/100, Attentive) and Arjun needed the most attention (63.43/100, Inattentive), driven mainly by a weak Night shift — showing the scoring model correctly separates safe driving from risky driving.

More than just a report generator, this project shows how simple sensor data can be turned into meaningful, actionable safety decisions — replacing guesswork with a consistent, repeatable scoring method that anyone can run in a few seconds.